

Fast Computation of Multivariate Subresultants

A report for the CCA Project #13

Yunus GÜRLEK

Frédéric XIA

Supervisor:

Weijia WANG

May 24, 2026

Contents

1	Introduction	2
2	Preliminaries	3
2.1	Subresultant polynomials	3
2.2	Structure theorem for subresultants	4
3	Reduction properties of subresultants	6
3.1	Specialization properties of subresultants	6
3.2	Modular reduction of subresultants	7
3.3	Reduction properties of subresultant polynomial remainders	8
4	Algorithms	8
4.1	Algorithm for bivariate subresultants	8
4.2	Generalization to trivariate and multivariate subresultants	9
5	Benchmarks	10
5.1	Benchmarks for integer polynomials	10
5.2	Benchmarks for rational polynomials	12
5.3	Comparison with Wolfram	13
6	Conclusion	13

1 Introduction

Let R be a ring, and let $f, g \in R[x]$ be two non-zero univariate polynomials of degree p and q , respectively, with $p > q$. When R is a domain, we denote by $\mathbb{K}$ its fraction field, and by $\gcd(f, g)$ the monic greatest common divisor of f and g in $\mathbb{K}[x]$.

To compute $\gcd(f, g)$, a common approach is to use the Euclidean algorithm, which consists of computing the sequence of remainders $r_0 = f$, $r_1 = g$, and $r_{i+1} = r_{i-1} \bmod r_i$ until we reach a remainder that is zero. The last non-zero remainder is then the greatest common divisor of f and g . However, this algorithm can be inefficient, as the size of the coefficients of the remainders can grow rapidly, leading to a large number of bit operations.

Example 1. Let $f = x^8 + x^6 - 3x^4 - 3x^3 + 8x^2 + 2x - 5$ and $g = 3x^6 + 5x^4 - 4x^2 - 9x + 21$. Applying the Euclidean algorithm with $r_0 = f$, $r_1 = g$, we have the following remainder sequence:

$$\begin{aligned} r_2 &= -\frac{5}{9}x^4 + \frac{1}{9}x^2 - \frac{1}{3}, & r_3 &= -\frac{117}{25}x^2 - 9x + \frac{441}{25} \\ r_4 &= \frac{233150}{19773}x - \frac{102500}{6591}, & r_5 &= -\frac{1288744821}{543589225} \end{aligned}$$

Thus, we have $\gcd(f, g) = 1$.

The subresultant theory, developed in the 19th century, offers a more efficient method for computing $\gcd(f, g)$. The subresultant polynomials $(\text{sResP}_i(f, g))_{0 \leq i \leq q}$, defined over $R[x]$, have the key property that an appropriately chosen subsequence is proportional to the remainder sequence produced by the Euclidean algorithm, as can be seen in the following example.

Example 1 (continued). The subsequence of subresultant polynomials is given by:

$$\begin{aligned} \text{sResP}_5(f, g) &= 15x^4 - 3x^2 + 9, & \text{sResP}_3(f, g) &= 65x^2 + 125x - 245, \\ \text{sResP}_1(f, g) &= 9326x - 12300, & \text{sResP}_0(f, g) &= 260708. \end{aligned}$$

Indeed, the bit size of the coefficients is better controlled, and we still have $\gcd(f, g) = 1$. Moreover, the subresultant polynomials not appearing in the subsequence are equal, up to scaling factors, to some subresultant polynomials in the subsequence:

$$\text{sResP}_4(f, g) = 25x^4 - 5x^2 + 15, \quad \text{sResP}_2(f, g) = 169x^2 + 325x - 637.$$

The relation between subresultant polynomials and the remainder sequence of the Euclidean algorithm was already known to Euler [7], while its use in the context of computer algebra was first proposed by Collins in 1967 [4]. Since then, subresultant-based algorithms have been widely studied and implemented in computer algebra systems and libraries, such as Wolfram, Maple, and FLINT.

State of the Art. Subresultant polynomials can be computed through various methods, the most efficient of which rely on the subresultant algorithm (see, e.g., [1, 3, 8]). Since its original development, refinements by Lazard and Ducos have led to an improved version, commonly referred to as the *improved* subresultant algorithm [6]. The algorithm achieves a bit complexity that is quadratic in both p and the bit size of the coefficients of the input polynomials.

When R is a field, or an integral domain with an effective fraction field, the specialization property of the resultant $\text{Res}(f, g) := \text{sResP}_0(f, g)$ enables the use of evaluation-interpolation techniques. Building on this observation, Collins's algorithm [5] applies multi-modular methods to compute the resultant efficiently, while relying on pseudo-remainder sequences rather than subresultant polynomials.

In practice, the improved subresultant algorithm is the default method for resultant computation in FLINT. Maple also relies on subresultant-based methods, supplemented by multi-modular techniques for polynomials of high degree. However, neither software provides a direct interface for computing subresultant polynomials. In contrast, Wolfram offers explicit access to subresultant polynomials via the `SubresultantPolynomials` function.

Contributions. In this report, we design and implement a fast algorithm for computing the resultant and subresultant sequence of two multivariate polynomials over $\mathbb{Q}$, $\mathbb{Z}$, or $\mathbb{F}_p$. The approach leverages the specialization property of subresultant polynomials and can be viewed as a generalization of Collins’s algorithm for computing resultants. Our implementation in FLINT demonstrates significant performance improvements over existing approaches in Wolfram, Maple, and FLINT itself, achieving speedups of up to 7x for the resultant of bivariate polynomials with exponents of bit size 6 and coefficients of bit size 50. We discuss the algorithmic design, implementation details, performance benchmarks, and potential avenues for further optimization in the subsequent sections of this report.

All implementations, as well as this research paper and findings are available in the following GitHub repository: <https://github.com/yunus433/cca-projet-2026>.

2 Preliminaries

Throughout the section, we let R be a ring, and let $f, g \in R[x]$ be two non-zero univariate polynomials of degree p and q , respectively. Thus, we can write $f = a_p x^p + a_{p-1} x^{p-1} + \cdots + a_0$ and $g = b_q x^q + b_{q-1} x^{q-1} + \cdots + b_0$, with $a_p, b_q \neq 0$.

2.1 Subresultant polynomials

Definition 2 (Sylvester matrix). *The Sylvester matrix associated with f and g is the $(p+q) \times (p+q)$ matrix defined as follows:*

$$\text{Syl}(f, g) = \begin{pmatrix} a_p & & & & b_q & & & \\ a_{p-1} & a_p & & & b_{q-1} & b_q & & \\ \vdots & \vdots & \ddots & & \vdots & \vdots & \ddots & \\ a_0 & a_1 & \cdots & a_p & b_0 & b_1 & \cdots & b_q \\ & a_0 & \ddots & \vdots & & b_0 & \ddots & \vdots \\ & & \ddots & a_1 & & & \ddots & b_1 \\ & & & a_0 & & & & b_0 \end{pmatrix},$$

where all unspecified entries are zero.

In Sylvester’s original paper [9], no explicit row or column convention was specified for the Sylvester matrix. Consequently, some sources in the literature (e.g., [2]) present the Sylvester matrix as the transpose of the form given above. In this report, we adopt a column-oriented convention, under which the Sylvester matrix is the matrix of the linear map

$$\begin{aligned} \phi : R[x]_{<q} \oplus R[x]_{<p} &\rightarrow R[x]_{<p+q} \\ (u, v) &\mapsto uf + vg \end{aligned}$$

in the canonical monomial basis $(1, x, x^2, \dots)$. This leads to the well-known definition of the resultant as the determinant of the Sylvester matrix.

Definition 3 (Resultant). *The resultant of f and g , denoted by $\text{Res}(f, g)$, is defined as the determinant of the Sylvester matrix $\text{Syl}(f, g)$.*

As for the subresultant polynomials, they are defined using the determinants of certain submatrices of the Sylvester matrix, called the *Sylvester-Habicht matrices*, which are defined as follows.

Definition 4 (Sylvester-Habicht matrix). *Given $0 \leq i \leq q$, the i -th Sylvester-Habicht matrix $S_i(f, g)$ is the $(p + q - i) \times (p + q - 2i)$ submatrix of $\text{Syl}(f, g)$ obtained as follows: take the first $p - i$ columns of $\text{Syl}(f, g)$ without their last i zero rows, then append the next $q - i$ columns starting from the $(p + 1)$ -th row of $\text{Syl}(f, g)$ without their last i zero rows.*

For $0 \leq j \leq i$, we denote by $M_{i,j}(f, g)$ the $(p + q - 2i) \times (p + q - 2i)$ submatrix of $S_i(f, g)$ obtained by taking the first $p + q - 2i - 1$ rows of $S_i(f, g)$ and the $(p + q - i - j)$ -th row of $S_i(f, g)$. Then, $M_{i,i}(f, g)$ is the matrix of the linear map

$$\begin{aligned} \phi_i : R[x]_{< q-i} \oplus R[x]_{< p-i} &\rightarrow R[x]_{< p+q-2i} \\ (u, v) &\mapsto uf + vg \end{aligned}$$

in the canonical monomial basis. In the same spirit as for resultants, the subresultants and subresultant polynomials of f and g are defined using the determinants of $M_{i,j}(f, g)$.

Definition 5 (Subresultant). *Given $0 \leq i \leq q$, the i -th subresultant of f and g , denoted by $\text{sRes}_i(f, g)$, is defined as the determinant of the matrix $M_{i,i}(f, g)$.*

Definition 6 (Subresultant polynomial). *Given $0 \leq i \leq q$, the i -th subresultant polynomial of f and g , denoted by $\text{sResP}_i(f, g)$, is defined as*

$$\text{sResP}_i(f, g) = \sum_{j=0}^i \det(M_{i,j}(f, g)) x^j.$$

We say that $\text{sResP}_i(f, g)$ is non-defective if $\deg(\text{sResP}_i(f, g)) = i$ (i.e., $\text{sRes}_i(f, g) \neq 0$), and defective otherwise.

2.2 Structure theorem for subresultants

While subresultants are defined as the determinants of certain submatrices of the Sylvester matrix, they also have a nice structure that relates them to the GCD and the remainders in the Euclidean algorithm, allowing us to compute them iteratively without performing any division in the fraction field of R . This is known as the structure theorem for subresultants, which is stated as follows.

Theorem 7 ([2, Thm. 8.56]). *Let $0 \leq i \leq q$. Suppose that $\text{sResP}_{i-1}(f, g)$ is non-zero and of degree $0 \leq j \leq i - 1$. Then,*

- *If $\text{sResP}_{j-1}(f, g)$ is zero, then $\text{sResP}_{i-1}(f, g) \propto \gcd(f, g)$.
Moreover, $\text{sResP}_k(f, g) = 0$ for all $k \leq j - 1$.*
- *If $\text{sResP}_{j-1}(f, g)$ is non-zero and of degree $0 \leq k \leq j - 1$, then $\text{sResP}_{k-1}(f, g)$ is proportional to $\text{Rem}(\text{sResP}_{i-1}(f, g), \text{sResP}_{j-1}(f, g))$.
Moreover, if $\text{sResP}_{j-1}(f, g)$ is defective, then $\text{sResP}_k(f, g) \propto \text{sResP}_{j-1}(f, g)$, and for all $k + 1 \leq \ell \leq j - 2$, $\text{sResP}_\ell(f, g) = 0$.*

An immediate consequence is that non-defective subresultant polynomials $\text{sResP}_{k-1}(f, g)$, as well as the $\text{sResP}_k(f, g)$, are essentially the remainders in the Euclidean algorithm, up to a scaling factor. The latter sequence of subresultant polynomials, shown in Example 1, is known as the *subresultant polynomial remainder sequence* of f and g .

We refer the reader to [2, Chapter 8] for a detailed proof of the structure theorem for subresultants, as well as the explicit formulas for the scaling factors.

The subresultant algorithm. The structure theorem for subresultants provides a way to compute the sequence of subresultant polynomials, and hence the GCD and resultant of two polynomials, without performing any division in the fraction field of R . Algorithm 1 gives the pseudo-code of the subresultant algorithm, with Lazard's optimization for computing the scaling factors [6].

Algorithm 1 SubresultantPolynomials

Require: $f, g \in R[x]$, $p = \deg(f)$, $q = \deg(g)$, $p > q$

Ensure: $(\text{sResP}_i(f, g))_{0 \leq i \leq q}$

```

1:  $\text{sResP}_p \leftarrow f$ 
2:  $s_p \leftarrow 1$ ,  $t_p \leftarrow 1$ 
3:  $\text{sResP}_{p-1} \leftarrow g$ 
4:  $t_{p-1} \leftarrow \text{lc}(\text{sResP}_{p-1})$ 
5:  $\text{sResP}_q \leftarrow (-1)^{(p-q)(p-q-1)/2} \text{lc}(\text{sResP}_{p-1})^{p-q-1} \text{sResP}_{p-1}$ 
6:  $s_q \leftarrow \text{lc}(\text{sResP}_q)$ 
7:  $i \leftarrow p + 1$ ,  $j \leftarrow p$ 
8: while  $\text{sResP}_{j-1} \neq 0$  do
9:    $k \leftarrow \deg(\text{sResP}_{j-1})$ 
10:  // Lazard's optimization for computing  $s_k = \text{sResP}_k(f, g)$ 
11:  for  $\delta = 1$  to  $j - k - 1$  do
12:     $t_{j-\delta-1} \leftarrow (-1)^\delta (t_{j-1} t_{j-\delta}) / s_j$ 
13:  end for
14:   $s_k \leftarrow t_k$ 
15:  // Computation of defective subresultant polynomials
16:   $\text{sResP}_k \leftarrow s_k \text{sResP}_{j-1} / t_{j-1}$ 
17:  for  $\ell = k + 1$  to  $j - 1$  do
18:     $\text{sResP}_\ell \leftarrow 0$ 
19:  end for
20:  // Computation of the next non-defective subresultant polynomial
21:   $\text{sResP}_{k-1} \leftarrow -\text{Rem}(s_k t_{j-1} \text{sResP}_{i-1}, \text{sResP}_{j-1}) / (s_j t_{i-1})$ 
22:   $t_{k-1} \leftarrow \text{lc}(\text{sResP}_{k-1})$ 
23:   $i \leftarrow j$ ,  $j \leftarrow k$ 
24: end while
25: for  $\ell = 0$  to  $j - 2$  do
26:   $\text{sResP}_\ell \leftarrow 0$ 
27: end for
28: return  $(\text{sResP}_i)_{0 \leq i \leq q}$ 

```

We emphasize that for the subresultant polynomials to be well-defined in $R[x]$, the assumption that $p > q$ is crucial. In the case of $p = q$, the algorithm defines $\text{sResP}_q(f, g) = b_q^{-1}g$, which may fail to lie in $R[x]$ if b_q is not a unit. Nevertheless, the polynomials $\text{sResP}_i(f, g)$ for $0 \leq i \leq q - 1$ remain well-defined in $R[x]$ and can be computed by the algorithm; we do not pursue the case of $p = q$ in this report for simplicity.

In addition, we remark that Algorithm 1 returns the sequence of subresultant polynomials $(\text{sResP}_i(f, g))_{0 \leq i \leq q}$, which contains all non-defective subresultant polynomials together with some defective ones. If only the non-defective subresultant polynomials are required, the computation of defective subresultant polynomials on Lines 15–19 can be omitted. Similarly, returning only $\text{sResP}_k(f, g)$ computed on Line 16 produces the subresultant polynomial remainder sequence of f and g .

3 Reduction properties of subresultants

Let $n \geq 2$, and $f, g \in R[x_1, \dots, x_n]$, where R is either $\mathbb{Z}$, $\mathbb{Q}$, or $\mathbb{F}_p$. f and g can be viewed as univariate polynomials in x_1 with coefficients in $R[x_2, \dots, x_n]$. The resultant of f and g with respect to x_1 , denoted by $\text{Res}_{x_1}(f, g)$, is defined as the resultant of these univariate polynomials in x_1 . Equivalently, it is the 0-th subresultant $\text{sRes}_{0, x_1}(f, g)$. The higher subresultants are defined analogously.

While subresultant algorithms can be used to compute $\text{Res}_{x_1}(f, g)$ (resp. the subresultants), their efficiency may be compromised by the rapid growth of the coefficients in $R[x_2, \dots, x_n]$ during the computation. To mitigate this, one may instead evaluate the input polynomials at suitable values of $x_2, \dots, x_n$, compute the resultant of the resulting univariate polynomials in x_1 , and then interpolate to recover $\text{Res}_{x_1}(f, g)$. This evaluation-interpolation approach replaces expensive symbolic computations in $R[x_2, \dots, x_n]$ with more efficient computations in R , often leading to significant performance improvements.

Still, it is crucial to ensure that the evaluation points are chosen such that the interpolation process is valid and yields the correct result. In the following, we discuss the conditions under which this approach is valid and how to choose suitable evaluation points.

3.1 Specialization properties of subresultants

Intuitively, let $\eta \in R^{n-1}$, and $f(\eta), g(\eta) \in R[x_1]$ be the univariate polynomials obtained by evaluating f and g at η . We then have $\text{Res}_{x_1}(f(\eta), g(\eta)) \in R$. We would like to have $\text{Res}_{x_1}(f(\eta), g(\eta)) = \text{Res}_{x_1}(f, g)(\eta)$, so that we can compute $\text{Res}_{x_1}(f, g)$ by interpolation. However, this equality does not hold for all η :

Example 8. Let $f = x_1x_2 + 1$ and $g = x_1 + 1$. Then, $\text{Res}_{x_1}(f, g) = x_2 - 1$. However, for $\eta = 0$, we have $\text{Res}_{x_1}(f(\eta), g(\eta)) = 1 \neq \text{Res}_{x_1}(f, g)(\eta) = -1$.

What happens in the above example is that the degree of f and g changes after evaluation, which leads to Sylvester matrix of different sizes. For the equality $\text{Res}_{x_1}(f(\eta), g(\eta)) = \text{Res}_{x_1}(f, g)(\eta)$ to hold, we should ensure that the degree of f and g does not change after evaluation. Formally, we have the following result.

Proposition 9. Let $f, g \in R[x_1, \dots, x_n]$, $\eta \in R^{n-1}$, and $h = \text{lc}_{x_1}(f)\text{lc}_{x_1}(g) \in R[x_2, \dots, x_n]$ be the product of the leading coefficients of f and g with respect to x_1 . If $h(\eta) \neq 0$, then $\text{Res}_{x_1}(f(\eta), g(\eta)) = \text{Res}_{x_1}(f, g)(\eta)$.

Proof. $h(\eta) \neq 0$ implies that $\text{lc}_{x_1}(f)(\eta) \neq 0$ and $\text{lc}_{x_1}(g)(\eta) \neq 0$. Therefore, $\deg_{x_1}(f(\eta)) = \deg_{x_1}(f)$ and $\deg_{x_1}(g(\eta)) = \deg_{x_1}(g)$, so that $\text{Syl}(f(\eta), g(\eta)) = \text{Syl}(f, g)(\eta)$, hence the result. $\square$

When R is either $\mathbb{Z}$ or $\mathbb{Q}$, the property that $h \neq 0$ is a generic condition, as $\{\eta \in \mathbb{C}^{n-1} : h(\eta) = 0\}$ is a proper Zariski-closed subset of $\mathbb{C}^{n-1}$. Consequently, we can find as many evaluation points as needed such that h does not vanish at these points, and then interpolate to recover $\text{Res}_{x_1}(f, g)$. However, it is not the case when $R = \mathbb{F}_p$ for whatever p . A natural

question is how many evaluation points we need to recover $\text{Res}_{x_1}(f, g)$ by interpolation. This is answered by the following degree bound of the resultant.

Proposition 10. *Let $f, g \in R[x_1, \dots, x_n]$. Denote by $\deg_{x_2, \dots, x_n}(f)$ (resp. $\deg_{x_2, \dots, x_n}(g)$) the total degree of f (resp. g) with respect to $x_2, \dots, x_n$. Then, we have*

$$\deg(\text{Res}_{x_1}(f, g)) \leq D := \deg_{x_1}(f) \deg_{x_2, \dots, x_n}(g) + \deg_{x_1}(g) \deg_{x_2, \dots, x_n}(f).$$

Proof. Each term in $\det(\text{Syl}(f, g))$ is a product of $\deg_{x_1}(g)$ coefficients of f and $\deg_{x_1}(f)$ coefficients of g , and therefore has degree bounded by D . We can then conclude by the definition of the resultant as the determinant of the Sylvester matrix. $\square$

Hence, we can recover $\text{Res}_{x_1}(f, g)$ by multivariate interpolation from at most $\binom{D+n-1}{n-1}$ evaluation points, thus from at most $D+1$ evaluation points when $n = 2$. When $R = \mathbb{F}_p$, h can vanish at at most $p^{n-2} \deg(h)$ points, so that when p is large enough such that $p^{n-2}(p - \deg(h)) > \binom{D+n-1}{n-1}$, we can still recover $\text{Res}_{x_1}(f, g)$ by interpolation.

Similarly, for the subresultant polynomials, as each coefficient of $\text{sResP}_i(f, g)$ is the determinant of a submatrix of the Sylvester matrix, we have the same specialization property as the resultant, and the same condition on the evaluation points.

Proposition 11. *Let $f, g \in R[x_1, \dots, x_n]$ with $\deg_{x_1}(f) > \deg_{x_1}(g) \geq 0$, $\eta \in R^{n-1}$, and $h = \text{lc}_{x_1}(f) \text{lc}_{x_1}(g) \in R[x_2, \dots, x_n]$. If $h(\eta) \neq 0$, then $\text{sResP}_i(f(\eta), g(\eta)) = \text{sResP}_i(f, g)(\eta)$ for all $0 \leq i \leq \deg_{x_1}(g)$.*

Proof. We already know that when $h(\eta) \neq 0$, we have $\text{Syl}(f(\eta), g(\eta)) = \text{Syl}(f, g)(\eta)$. Hence, we have $M_{i,j}(f(\eta), g(\eta)) = M_{i,j}(f, g)(\eta)$ for all $0 \leq j \leq i \leq \deg_{x_1}(g)$, so that $\text{sResP}_i(f(\eta), g(\eta)) = \text{sResP}_i(f, g)(\eta)$ for all $0 \leq i \leq \deg_{x_1}(g)$. $\square$

To compute $\text{sResP}_i(f, g)$, we can then compute $\text{sResP}_i(f(\eta), g(\eta))$ for sufficiently many evaluation points η , and then interpolate to recover each coefficient of $\text{sResP}_i(f, g)$. The number of evaluation points needed is bounded by the same bound as the resultant, as each coefficient of $\text{sResP}_i(f, g)$ is a determinant of a submatrix of the Sylvester matrix, and therefore has degree bounded by D .

3.2 Modular reduction of subresultants

Let $R = \mathbb{Z}$ or $\mathbb{Q}$, and $f, g \in R[x_1, \dots, x_n]$. Instead of applying the evaluation-interpolation approach directly to f and g , we can first reduce f and g modulo a prime p , and then apply the evaluation-interpolation approach to $f \bmod p$ and $g \bmod p$. This is particularly useful when the coefficients of f and g are large, as it allows us to perform more efficient computations in $\mathbb{F}_p$ instead of $\mathbb{Z}$ or $\mathbb{Q}$. However, the choice of prime p is not arbitrary:

Example 12. *Let $f = 3x_1^2 + 2x_1x_2 + 1$, $g = x_1 + 1$, and $p = 3$. Then, $\text{Res}_{x_1}(f, g) \bmod p = 1 + x_2$, and $\text{Res}_{x_1}(f \bmod p, g \bmod p) = 2 + 2x_2 \neq \text{Res}_{x_1}(f, g) \bmod p$.*

Proposition 13. *Let $f, g \in \mathbb{Z}[x_1, \dots, x_n]$, $h = \text{lc}_{x_1}(f) \text{lc}_{x_1}(g)$, and p be a prime. If $h \bmod p \neq 0$, then $\text{Res}_{x_1}(f \bmod p, g \bmod p) = \text{Res}_{x_1}(f, g) \bmod p$.*

Proof. $h \bmod p \neq 0$ implies that $\text{lc}_{x_1}(f) \bmod p \neq 0$ and $\text{lc}_{x_1}(g) \bmod p \neq 0$. Therefore, $\deg_{x_1}(f \bmod p) = \deg_{x_1}(f)$, $\deg_{x_1}(g \bmod p) = \deg_{x_1}(g)$, and $\text{Syl}(f \bmod p, g \bmod p) = \text{Syl}(f, g) \bmod p$, hence the result. $\square$

Similarly, we have the same modular reduction property for the subresultant polynomials, with the same condition on p .

When $n = 2$, a prime p between 2^{30} and $2^{31} - 1$ is large enough to ensure sufficiently many evaluation points for interpolation in practice, while still being small enough to fit the machine word size, so that the computation of $\text{Res}_{x_1}(f \bmod p, g \bmod p)$ (resp. $\text{sResP}_i(f \bmod p, g \bmod p)$) can be done efficiently in $\mathbb{F}_p$.

3.3 Reduction properties of subresultant polynomial remainders

In Algorithm 1, returning the subsequence $\text{sResP}_k(f, g)$ computed on Line 16 yields the subresultant polynomial remainder sequence of f and g . One immediate application of this variant is the computation of the Sturm–Habicht sequence of f , namely the subresultant polynomial remainder sequence of f and its derivative $\frac{\partial f}{\partial x_1}$, up to sign changes.

A natural question is whether we can compute the subresultant polynomial remainders with evaluation-interpolation as well. The answer is positive, but one needs to pay special attention to the choice of evaluation points.

Example 14. Let $f = x_1^4 + x_1^2 x_2 + 1$ and $g = x_1^3 + x_1 + x_2$. Then, the subresultant polynomial remainders of f and g with respect to x_1 are

$$\begin{aligned} \text{sResP}_0(f, g) &= x_2^5 - x_2^4 - 2x_2^3 + 5x_2^2 - 4x_2 + 4, \quad \text{sResP}_1(f, g) = (2x_2^2 - 3x_2 + 2)x_1 + x_2^3 - 2x_2^2, \\ \text{sResP}_2(f, g) &= (x_2 - 1)x_1^2 - x_2 x_1 + 1. \end{aligned}$$

However, $\text{sResP}_2(f, g)$ becomes defective after evaluating at $x_2 = 1$.

Hence, suitable evaluation points for computing subresultant polynomial remainders are those that do not make any non-defective subresultant polynomial defective.

While this condition is likewise generic, it is less straightforward to verify than the condition for the resultant, where it suffices to check whether $\text{lc}_{x_1}(f) \text{lc}_{x_1}(g)$ vanishes at the evaluation points. Nevertheless, the number of subresultant polynomial remainders, as well as their degrees, can be determined by evaluating at a sufficiently large set of random points and inspecting the degree of the resulting subresultant polynomials. One can then select evaluation points for which the observed degree sequence matches the expected one, and finally reconstruct the polynomials via interpolation.

4 Algorithms

In this section, we propose algorithms to efficiently compute the subresultant polynomials of two polynomials $f, g \in R[x_1, \dots, x_n]$ with coefficients in $R = \mathbb{Z}, \mathbb{Q}$, or $\mathbb{F}_p$ with sufficiently large p , by using an evaluation-interpolation approach. We recall the results from the previous section:

1. The subresultants of two evaluated polynomials are equal to the subresultants evaluated at the same point, outside the set of points where either $\text{lc}_{x_1}(f)$ or $\text{lc}_{x_1}(g)$ vanishes.
2. The degree bound for subresultants is $\deg_{x_1}(f) \deg_{x_2, \dots, x_n}(g) + \deg_{x_1}(g) \deg_{x_2, \dots, x_n}(f)$.

4.1 Algorithm for bivariate subresultants

For bivariate polynomials $f, g \in R[x_1, x_2]$, the degree bound of their resultant is given by $\deg_{x_1}(f) \deg_{x_2}(g) + \deg_{x_1}(g) \deg_{x_2}(f)$. Using the above facts, Algorithm 2 computes the subresultant polynomials of f and g by evaluating f and g at sufficiently many points, computing

the subresultant polynomials of the evaluated univariate polynomials, and then interpolating to recover the subresultant polynomials of f and g . In Algorithm 2, we denote by $\text{Interpolate}(x, y)$ the interpolation method from the set of points $(x_i, y_i)_{1 \leq i \leq N+1}$, which is provided by FLINT in our implementation.

Algorithm 2 BivariateSubresultantPolynomials

Require: $f, g \in R[x_1, x_2]$, $p = \deg_{x_1}(f)$, $q = \deg_{x_1}(g)$, $p > q$
Ensure: $(\text{sResP}_i(f, g))_{0 \leq i \leq q}$

```

1:  $N \leftarrow \deg_{x_1}(f) \deg_{x_2}(g) + \deg_{x_1}(g) \deg_{x_2}(f)$ 
2: // We choose evaluation points starting from 0 and increasing,
3: // until we have  $N + 1$  points where leading coefficients do not vanish
4:  $\eta \leftarrow 0$ 
5: for  $i = 1$  to  $N + 1$  do
6:   while  $\text{lc}_{x_1}(f)(\eta) = 0$  or  $\text{lc}_{x_1}(g)(\eta) = 0$  do
7:      $\eta \leftarrow \eta + 1$ 
8:   end while
9:    $x^i \leftarrow \eta$ 
10:   $\eta \leftarrow \eta + 1$ 
11: end for
12: // We compute the subresultant polynomials specialized at the evaluation points
13: for  $i = 1$  to  $N + 1$  do
14:   for  $j = 0$  to  $q$  do
15:      $y_j^i \leftarrow \text{sResP}_j(f(x^i), g(x^i))$ 
16:   end for
17: end for
18: // We interpolate the coefficients to recover the subresultant polynomials
19: for  $j = 0$  to  $q$  do
20:    $\text{sResP}_j \leftarrow \text{Interpolate}(x, y_j)$ 
21: end for
22: return  $(\text{sResP}_j)_{0 \leq j \leq q}$ 

```

Note that interpolation points can also be chosen with another method, e.g. randomly on a given bit size or in the order of increasing bit size. For $R = \mathbb{Z}$, a good choice is to choose points in the order $0, 1, -1, 2, -2, \dots$, etc. We implement these different approaches in our work and compare their performance in the following sections.

4.2 Generalization to trivariate and multivariate subresultants

For trivariate polynomials $f, g \in R[x_1, x_2, x_3]$, we can apply the same evaluation-interpolation approach as in the bivariate case, but with a different choice of evaluation points. Specifically, we evaluate at points of the form (η_2, η_3) , chosen so that neither $\text{lc}_{x_1}(f)$ nor $\text{lc}_{x_1}(g)$ vanishes at these points, then perform multivariate interpolation to recover the subresultant polynomials of f and g . When fixing η_2 , however, one must also ensure that $\text{lc}_{x_1}(f)$ and $\text{lc}_{x_1}(g)$ are not identically zero as polynomials in x_3 .

The same idea extends recursively to polynomials in more than three variables, where we evaluate at points of the form $(\eta_2, \dots, \eta_n)$, chosen so that $\text{lc}_{x_1}(f)$ and $\text{lc}_{x_1}(g)$ do not vanish at these points.

5 Benchmarks

Here, we present some benchmarks comparing our implementation of the algorithm against the FLINT library for bivariate polynomial resultant computation.

All benchmarks presented below are performed on an Apple MacBook with 8-core Apple M2 SoC, 16 GB unified memory. All results are averages over 5 independent trials, and the bit size of polynomial coefficients is fixed to be 50 in all cases.

Note that for our proposed implementation, we present 6 different running times, based on the method used to select interpolation points:

1. Random Points: uniformly random points with the same bit size as the polynomial coefficients.
2. Random Positive Points: uniformly random positive points with the same bit size as the polynomial coefficients.
3. Small Random Points: random points with bit size comparable to the exponent bit size.
4. Small Random Positive Points: random positive points with bit size comparable to the exponent bit size.
5. Ordered Points: points chosen in increasing order of bit size.
6. Ordered Positive Points: positive points chosen in increasing order of bit size.

Finally, we note that even though we implement and test our algorithm for the subresultant sequence of polynomials as well, we do not present any benchmarks here, since FLINT does not expose subresultant or subresultant polynomial sequences, so it is not possible to give any comparison.

5.1 Benchmarks for integer polynomials

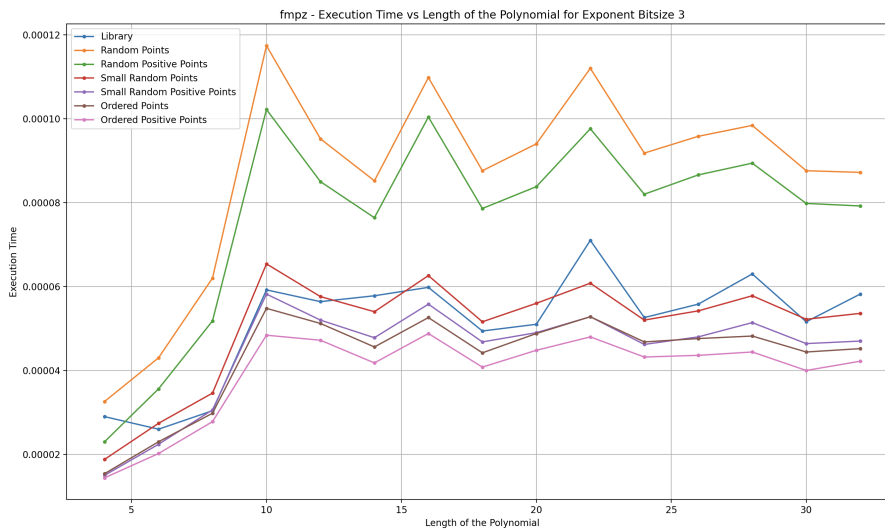
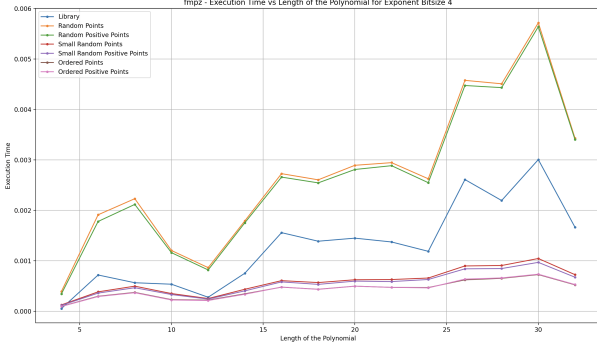


Figure 1: FMPZ bivariate polynomial resultant execution time (in seconds) for exponent bit size 3 with varying lengths.

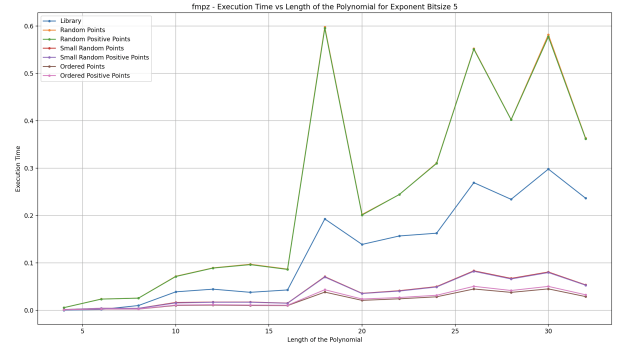
In Figure 1, we first compare our implementation with FLINT in small polynomials, i.e. of exponent bit size 3. We see that while the random selection of points for the interpolation

method is significantly slower than the rest, other methods have similar running times. This important difference between random selection and others can be explained by the bit size of the points used in the interpolation. When a uniformly random selection is performed, the bit size of the points used in the interpolation is the same as the bit size of coefficients, i.e. 50. Thus, the interpolation takes significantly more time than other approaches.

For the remaining methods, another important observation is that even though FLINT's implementation is slower in some cases, there is no significant difference between their running times.



(a) FMPZ bivariate polynomial resultant execution time (in seconds) for exponent bit size 4 with varying lengths.



(b) FMPZ bivariate polynomial resultant execution time (in seconds) for exponent bit size 5 with varying lengths.

In Figures 2a and 2b, we extend our analysis to bigger polynomials with the exponent bit size 4 and 5, respectively. We see that the relation between the random selection method and other categories is still the same, while the difference between the FLINT library implementation and our methods starts to become visible. As the length of polynomials increase, FLINT performs much slower than our algorithm. Notice also that there is not too much difference between randomly selecting interpolation points with small bit size and having them ordered in the bit size, which supports our intuition that the running time of polynomial interpolation depends on the bit size of selected points.

Moreover, we can observe that all curves follow a similar shape with considerable fluctuations. Even though the overall trend is that increasing the polynomial's length increases the running time, we see that for some lengths there is a sharp decrease in the measured time. This result can be explained by the properties of the random bivariate polynomial generation function of FLINT. For some testcases, the generated polynomials have a resultant that is much harder to compute. For instance, when we generate random bivariate polynomials in FLINT with parameters of length 10, exponent bitsize 5, and coefficient bitsize 8, we may get the following two polynomials:

$$f_1 = 3x^5 + x^3y^5 - 63x^2y + xy^3 - 14y^5 + 11y$$

$$f_2 = x^{15}y^{11} + x^{15} + 45x^{12}y^8 + 55x^4y + 3x^3y + 226x^2 - 55xy^4 + 9x + 31$$

Both of these two polynomials are generated by the exact same function call, but they differ a lot in terms of arithmetic operations while computing their resultant. Thus, the random polynomial generation algorithm of FLINT cannot be efficiently used to have a more stable benchmark. A better random polynomial generation algorithm can theoretically be used, which we provide an example of for univariate polynomials in our work. We do not extend our simple implementation to random generation of bivariate polynomials as it falls way out of scope of this paper.

These fluctuations could also be reduced by increasing the number of trials per length. However, increasing number of trials also increase the total benchmarking time significantly,

and since our main goal is to compare the different methods rather than to estimate their precise dependence on the polynomial length, we keep this methodology.

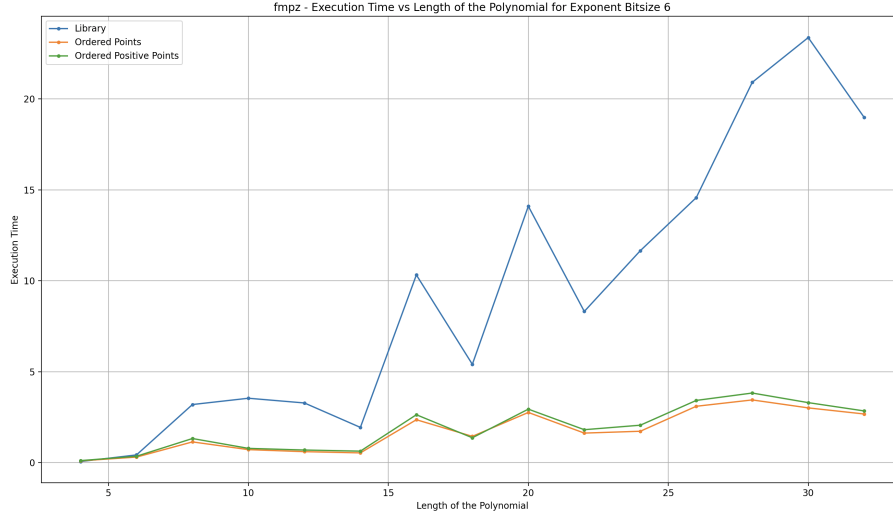
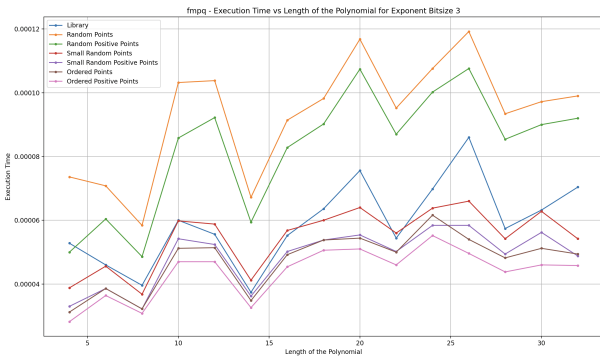


Figure 3: FMPZ bivariate polynomial resultant execution time (in seconds) for exponent bit size 6 with varying lengths.

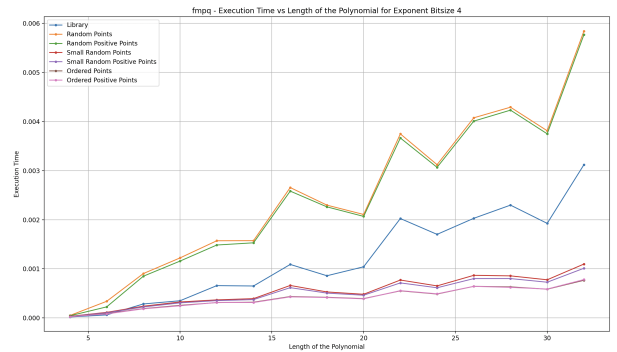
Finally, we measure with polynomials of the exponent bit size 6, where we only include the library and ordered implementations to minimize the total running time. We observe that the difference between both methods becomes even more important, where we achieve an approximate 7x speedup compared to FLINT. For bigger bit sizes, the total running time exceeds 30 seconds for a single measurement, thus we do not present the results here.

5.2 Benchmarks for rational polynomials

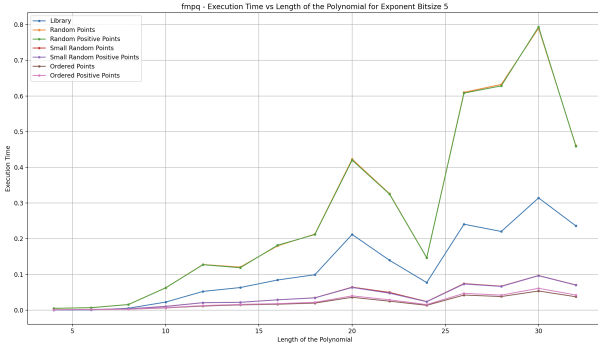
In this section, we extend our analysis to polynomials over $\mathbb{Q}$ instead of $\mathbb{Z}$, but note that the presented results do not differ substantially from what we have observed for $\mathbb{Z}$. Interpolation using random points with big bit size is significantly slower than the rest of the methods, and the FLINT implementation starts becoming much slower than our implementations as the exponent bit size and the length of the polynomials increase. Using ordered points in bit size for interpolation is slightly faster than using random points with small bit size, but both methods are still significantly faster than FLINT. The maximum achieved performance improvement is again around 7x compared to FLINT.



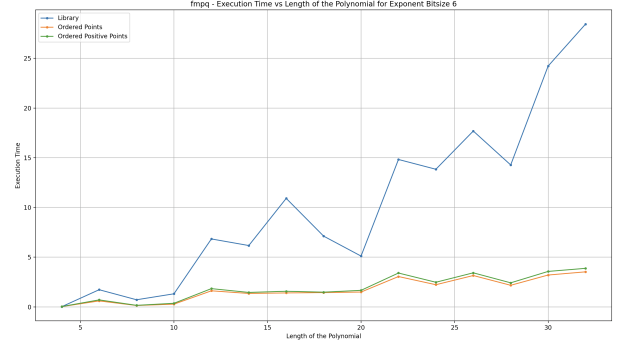
(a) FMPQ bivariate polynomial resultant execution time (in seconds) for exponent bit size 3 with varying lengths.



(b) FMPQ bivariate polynomial resultant execution time (in seconds) for exponent bit size 4 with varying lengths.



(a) FMPQ bivariate polynomial resultant execution time (in seconds) for exponent bit size 5 with varying lengths.



(b) FMPQ bivariate polynomial resultant execution time (in seconds) for exponent bit size 6 with varying lengths.

5.3 Comparison with Wolfram

We also perform an equivalent analysis to compare our fastest implementation (i.e. ordered points) against Wolfram for bivariate polynomial resultant computation in exponents of bit size. We see that even though Wolfram is significantly faster than FLINT, it is still slower than our implementation with a factor of 2.5x.

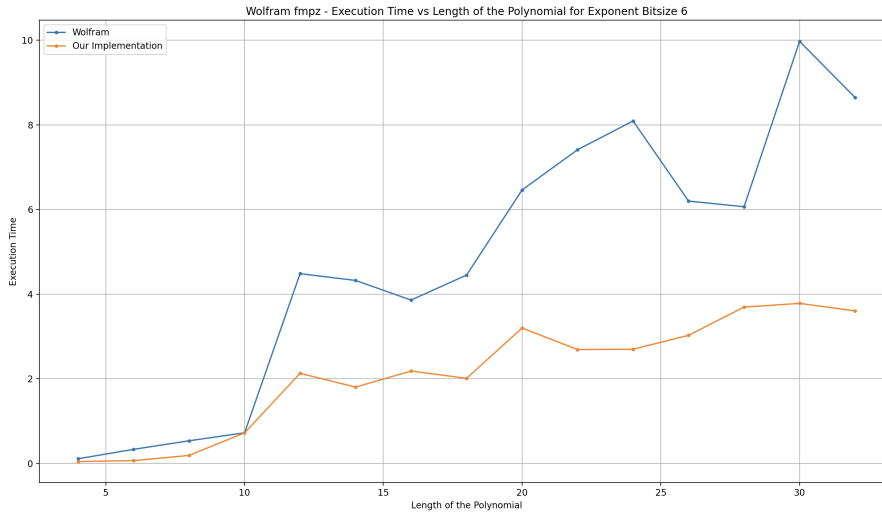


Figure 6: Wolfram - FMPZ bivariate polynomial resultant execution time (in seconds) for exponent bit size 6 with varying lengths.

6 Conclusion

We conclude that using an evaluation-interpolation approach for computing resultants, subresultants, or subresultant polynomials can be significantly faster than existing methods in well-known libraries such as FLINT. We observe a $7\times$ performance improvement in polynomials with exponent bit size 6 and coefficient bit size 50. We also note that the performance difference becomes more significant as the degree and length of polynomials increase.

We also observe that the method used to choose random points for the interpolation step of our algorithm has a strong effect on performance. In particular, the interpolation performs faster when working with points of smaller bitsize. Thus, we identify the fastest implementation as the one that chooses points in increasing order of bit size instead of using any randomization.

As possible next steps, we identify:

- Benchmarking the subresultant implementation for bivariate polynomials, using Wolfram or a similar system as a reference point,
- Extending the implementation from resultants and subresultants to subresultant polynomials,
- Extending the implementation to polynomials in higher dimensions, such as trivariate polynomials,
- Optimizing, testing, and benchmarking the implementation on polynomials of higher degree and length.

References

- [1] A. Akritas. *Elements of Computer Algebra with Applications*. John Wiley & Sons, Inc., 1989.
- [2] S. Basu, R. Pollack, and M.-F. Roy. *Algorithms in Real Algebraic Geometry*. Springer, 2006.
- [3] H. Cohen. *A Course in Computational Algebraic Number Theory*. Springer Science & Business Media, 2013.
- [4] G. E. Collins. Subresultants and reduced polynomial remainder sequences. *Journal of the ACM (JACM)*, 14(1):128–142, 1967.
- [5] G. E. Collins. The calculation of multivariate polynomial resultants. *Journal of the ACM (JACM)*, 18(4):515–532, 1971.
- [6] L. Ducos. Optimizations of the subresultant algorithm. *Journal of Pure and Applied Algebra*, 145(2):149–163, 2000.
- [7] L. Euler. Demonstration sur le nombre des points, ou deux lignes des ordres quelconques peuvent se couper. *Memoires de l'academie des sciences de Berlin*, pages 234–248, 1750.
- [8] K. O. Geddes, S. R. Czapor, and G. Labahn. *Algorithms for Computer Algebra*. Springer, 1992.
- [9] J. J. Sylvester. On a theory of the syzygetic relations of two rational integral functions, comprising an application to the theory of sturm's functions, and that of the greatest algebraical common measure. *Philosophical transactions of the Royal Society of London*, 143(18):407–548, 1853.